

1. 2's complement Addition and Subtraction:-

Given,

$$A = +45$$

$$B = -23$$

Binary conversion

$$+45 = 00101101$$

$$+23 = 00010111$$

$$-23 = 11101001 \text{ (2's complement)}$$

Addition: $45 + (-23)$

$$\begin{array}{r} 0010101 \\ + 11101001 \\ \hline 100010110 \end{array}$$

Ignore the carry:

$$00010110 = 22$$

Therefore :

$$45 + (-23) = 22$$

Subtraction: $45 - (-23)$

Subtraction is addition of 2's complement:-

$$45 - (-23) = 45 + 23$$

$$\begin{array}{r} 00101101 \\ + 00010111 \\ \hline 01000100 \end{array}$$

$$01000100 = 68$$

$$45 - (-23) = 68.$$

overflow:- No overflow occurs because the sign of the result is valid. in signed addition, overflow detected

4. bit carry look ahead Adder (CLA):-

Given,

Two binary numbers

$$A = A_3 A_2 A_1 A_0$$

$$B = B_3 B_2 B_1 B_0$$

1. Generate and propagate.

For each bit:-

Generate:-

$$G_i = A_i B_i$$

propagate:-

$$P_i = A_i \oplus B_i$$

carry calculation:-

$$C_1 = G_0 + P_0 C_0$$

$$C_2 = G_1 + P_1 G_0 + P_1 P_0 C_0$$

$$C_3 = G_2 + P_2 G_1 + P_2 P_1 G_0 + P_2 P_1 P_0 C_0$$

$$C_4 = G_3 + P_3 G_2 + P_3 P_2 G_1 + P_3 P_2 P_1 G_0 + P_3 P_2 P_1 P_0 C_0$$

Sum:-

$$S_i = P_i \oplus C_i$$

3. Time delay:-

Ripple carry Adder:

carry moves from one stage to the next.

$$C_0 \rightarrow C_1 \rightarrow C_2 \rightarrow C_3 \rightarrow C_4$$

So delay is more.

CLA:

All carries are calculated in advance and in parallel.

So delay is less.

4. conclusion:-

CLA is faster than ripple carry adder because it calculates carry signals in parallel instead of waiting for carry to ripple through each stage.

Therefore, CLA is used in high-speed processors to reduce addition time.

8. Booth's Algorithm:-

Given,

$$M = -6, Q = +5$$

use 4-bit representation.

$$-6 = 1010$$

$$+5 = 0101$$

Initial values:

$$\Rightarrow \text{Accumulator (A)} = 0000$$

$$\Rightarrow \text{Multiplier (Q)} = 0101$$

$$\Rightarrow \text{Multiplicand (M)} = 1010$$

$$\Rightarrow \text{Booth bit (Q-1)} = 0$$

Booth rules:-

$Q_0 Q_{-1}$	operation
00	No operation
11	No operation

01	$A = A + M$
10	$A = A - M$

After every operation, perform an arithmetic right shift of A, Q, Q_{-1} .

Step-1:

$$Q_0 Q_{-1} = 10$$

So,

$$A = A - M$$

$$0000 - 1010 = 0110$$

Arithmetic right shift:

$$A = 0011, Q = 0010, Q_{-1} = 1$$

Step-2:

$$Q_0 Q_{-1} = 01$$

So,

$$A = A + M$$

$$0011 + 1010 = 1101$$

Arithmetic right shift:

$$A = 1110, Q = 1001, Q_{-1} = 0$$

Step-3:

$$Q_0 Q_{-1} = 10$$

So,

$$A = A - M$$

$$1110 - 1010 = 0100$$

Arithmetic right shift:

$$A = 0010, Q = 0100, Q_{-1} = 1$$

Step-4:

$$Q_0 Q_{-1} = 01$$

So,

$$A = A + M$$

$$0010 + 1010 = 1100$$

Arithmetic right shift:

$$A = 1110, Q = 0010, Q_{-1} = 0$$

Combine A and Q

$$AQ = 1110010$$

$$1110010 = -30$$

$$\therefore -6 \times 5 = -30$$

4. Restoring and non-restoring division:-

Given,

$$13 \div 3$$

Binary:

$13 = 1101$, $3 = 0011$ then use 4-bit division.

1. restoring division:-

Initial:- $A = 00000$, $Q = 1101$, $M = 00011$.

Here:- $A = \text{accumulator}$, $Q = \text{dividend}$, $M = \text{divisor}$.

1. Shift left A, Q: $A = 00001$, $Q = 1010$

Subtract: $A = A - M$

$00001 - 00011 = 11110$; A is negative, so restore.

$11110 + 00011 = 00001$; Therefore, quotient bit = 0

$$A = 00001, Q = 1010$$

2. Shift $\rightarrow A = 0011$

$$A - M = 0011 - 0011 = 0000 \rightarrow Q_0 = 1$$

3. Shift $\rightarrow A = 0000$

$$A - M = 0000 - 0011 = -3 \rightarrow \text{restore} \rightarrow A = 0000, Q_0 = 0.$$

4. Shift $\rightarrow A = 0001$

$A - M = 0001 - 0011 = -2 \rightarrow$ restore $\rightarrow A = 0001, Q_0 = 0$

Therefore

$$Q = 0100 = 4$$

$$R = 0001 = 1$$

2. non-restoring division:-

initial:- $A = 0000, Q = 1101, M = 0011$

Rule:-

* A positive $\rightarrow$ subtract M

* A negative $\rightarrow$ add M

Steps:-

1. $A - M \rightarrow$ negative $\rightarrow Q_0 = 0$

2. $A + M \rightarrow$ positive $\rightarrow Q_0 = 1$

3. $A - M \rightarrow$ negative $\rightarrow Q_0 = 0$

4. $A + M \rightarrow$ negative $\rightarrow Q_0 = 0$

Therefore:

$$Q = 0100 = 4$$

$$R = 0001 = 1$$

comparison:-

restoring: negative remainder is restored every time $\rightarrow$ more operations

non-restoring: no immediate restoration $\rightarrow$ fewer operations and faster.

$$\therefore 13 \div 3 = 4 \text{ remainder } 1.$$

non restoring division is preferred because it reduces the number of operations and is faster.

5. IEEE 754 Floating point representation:-

Given,

$$-13.25$$

Convert to binary :- $13.25 = 1101.01_2$

$$\text{So, } -13.25 = -1101.01_2$$

normalize:

$$-1.10101 \times 2^3$$

1. Single precision - 32 bits:-

Fields:

$$* \text{Sign} = 1 \text{ (negative)}$$

$$* \text{Exponent} = 3 + 127 = 130$$

$$* 130 = 1000010_{10}$$

$$* \text{Mantissa} = 101010000000000000000000$$

Format:-

$$\text{Sign}(1) | \text{Exponent}(8) | \text{Mantissa}(23)$$

2. Double precision - 64 bits:-

Fields:-

$$* \text{Sign} = 1$$

$$* \text{Exponent} = 3 + 1023 = 1026$$

$$* 1026 = 1000000010_{10}$$

Format:-

$$\text{Sign}(1) | \text{Exponent}(11) | \text{Mantissa}(52)$$

3. Floating - point Addition:-

For example:-

$$13.25 + 5.5$$

Steps:-

1. convert both numbers to binary.
2. make their exponents are equal
3. Add mantissa.
4. normalize the result.
5. round the result to fill the available mantissa bits.

Problems:-

- * Normalization:- result must be in $1.x \times 2^n$ form.
- * Rounding:- Extra bits are removed when precision is limited.
- * Precision loss:- very small numbers may lose accuracy when added to very large numbers.

Conclusion:-

Single precision:- 32-bits $\rightarrow$ less precision.

Double precision:- 64-bits $\rightarrow$ more precision and less rounding error.

$\therefore$ double precision is more accurate than single precision.